

Proposta de um Sistema Multiagentes baseado em *LLM*: Integração em um Software de Gerenciamento e Otimização de Carregamento e Descarregamento de Navios

Antonio S. Neto, Rodrigo D. Macedo, Carolina S. Almeida *
Arley Henrique Duque ** Sandro Jerônimo de Almeida ***

* *DeepTera Sistemas Inteligentes*
{antonio, rodrigo, carolinajeronomo}@deeptera.com
** *Framework Digital* | arleyduque@frwk.com.br
*** *Pontifícia Universidade Católica de Minas Gerais*
sandro.j@pucminas.br

Abstract: The industry faces challenges that demand efficiency and new technologies to remain competitive. This work proposes a Multi-Agent System (MAS) based on LLM, integrated into a conventional software for managing ship loading and unloading, aiming to enhance user interaction. The MAS, composed of four specialized agents (Manager, Helper, Optimizer, and Data Analytics), utilizes LLM in real time and a knowledge base from the conventional system for contextualized decision-making. Use cases demonstrate its ability to register ships, generate optimized plans, and create personalized visual analyses. The interaction between agents, mediated by the Manager, resulted in gains in agility, automation, and insight generation, proving to be a promising solution for industrial challenges.

Resumo: A indústria enfrenta desafios que exigem eficiência e novas tecnologias para se manter competitiva. Este trabalho propõe um Sistema Multiagente (SMA) baseado em LLM, integrado a um software para gerenciamento de carregamento e descarregamento de navios, visando aprimorar a interação dos usuários. O SMA, composto por quatro agentes especializados (*Manager*, *Helper*, *Optimizer* e *Data Analytics*), utiliza LLM em tempo real e uma base de conhecimento do sistema convencional para decisões contextualizadas. Casos de uso demonstram sua capacidade em cadastrar navios, gerar planos otimizados e criar análises visuais personalizadas. A interação entre os agentes, mediada pelo *Manager*, resultou em ganhos em agilidade, automação e geração de insights, mostrando-se uma solução promissora para desafios industriais.

Keywords: Multi-Agent Systems; Large Language Models; Industrial Automation; Intelligent Agents; Artificial intelligence (AI).

Palavras-chaves: Sistemas Multiagentes; Modelos de Linguagem de Grande Escala; Automação Industrial; Agentes Inteligentes; Inteligência Artificial.

1. INTRODUÇÃO

A indústria contemporânea enfrenta um cenário de constantes transformações, impulsionada pela necessidade de se manter competitiva em um mercado globalizado. A busca por maior eficiência operacional, redução de custos, otimização de recursos e minimização de impactos ambientais tem se tornado um imperativo para empresas que desejam se destacar. Um exemplo emblemático é o gerenciamento do carregamento e descarregamento de navios, onde a sequencição e a logística desempenham um papel crucial. Ineficiências nesses processos podem levar a atrasos, aumento de custos e intensificação dos impactos ambientais causados pela água de lastro, já que quanto mais lastro for utilizado de forma desnecessária, maior será o prejuízo ao meio ambiente.

Nesse contexto, a adoção de tecnologias inovadoras, como o uso de modelos, métodos de otimização e decisão, visão computacional e a *Inteligência Artificial* (IA), tem ganhado destaque. Técnicas mais recentes, como *Grandes Modelos de Linguagem* (LLM)* e *Sistemas Multiagentes* (SMA), têm potencial para revolucionar a forma como processos industriais são gerenciados e executados (Chen et al., 2024). Essa evolução é parte de um movimento maior, que vai além da Indústria 4.0 — focada em tecnologias como *Internet das Coisas* (IoT), Big Data, computação em nuvem (Lasi et al., 2014) e Digital Twin — e avança para a Indústria 5.0, que prioriza a colaboração entre humanos e máquinas, produção personalizada, flexibilidade e manufatura inteligente (Leng et al., 2022; Paschek et al., 2022; Hassan et al., 2024), abrindo caminho para soluções mais ágeis e sustentáveis (Wang et al., 2024).

Considerando os desafios da indústria, principalmente da logística naval, e a necessidade de soluções tecnológicas,

* Do inglês: Large Language Models.

este trabalho propõe a implementação de um Sistema Multiagentes baseado em LLM que pode ser integrado a sistemas convencionais, agilizando a interação de usuários e a automação de tarefas nos processos envolvidos. Além da proposta de arquitetura de agentes inteligentes, o artigo relata a experiência prática de integração desses agentes com o software convencional Bullker¹, voltada para gerenciamento e otimização matemática de sequenciamento de tarefas no embarque e desembarque de navios graneleiros. São explorados os desafios e as soluções encontradas durante o desenvolvimento do SMA proposto e sua integração ao sistema Bullker.

O sistema multiagente proposto é composto por agentes, hierarquicamente organizados, que possuem funções especializadas. Esses agentes estão aptos a interagir com o usuário e funções do sistema convencional. As demandas apresentadas pelos usuários chegam para os agentes, que avaliam, realizam processamentos e retornam para o usuário resultados e informações. O processamento pode envolver a consulta aos dados e modelos da aplicação convencional; no caso deste trabalho, o sistema Bullker.

Com o SMA proposto o usuário pode utilizar funcionalidades do sistema Bullker a partir de interação em linguagem natural, tais como cadastros de navios e fretes, geração de planos otimizados e seguros de carregamento e descarregamento de navios e análise visual de dados. O que antes era feito de maneira estruturada, agora pode ser realizado com mais flexibilidade, naturalidade, agilidade e personalização. A experiência compartilhada neste trabalho poderá guiar profissionais e empresas que desejarem evoluir seus sistemas computacionais com abordagens de SMA e LLM.

Este trabalho está organizado da seguinte forma. Na Seção 2, são apresentados os trabalhos relacionados, abordando sistemas multiagentes, modelos LLM e suas aplicações na indústria. A Seção 3 descreve a arquitetura do sistema proposto, detalhando os agentes, prompts e suas funcionalidades, bem como a integração entre eles e o sistema convencional. A Seção 4 apresenta um caso de uso com exemplos de interação dos agentes e os resultados obtidos. Por fim, a Seção 5 traz as conclusões do trabalho, suas limitações e perspectivas para pesquisas futuras.

2. CONCEITOS E TRABALHOS RELACIONADOS

Na área de Inteligência artificial, dois campos têm ganhado relevância nos últimos anos: os SMA e os modelos de LLM, com potencial para aplicação em diversos segmentos, como na indústria.

Os SMA são um caso particular de sistemas distribuídos, cuja particularidade reside na autonomia e no comportamento individualista de seus componentes, conhecidos como agentes, que buscam satisfazer seus próprios objetivos (Wooldridge, 2009). Um agente inteligente deve ser capaz de perceber seu ambiente, raciocinar sobre como atingir seus objetivos, agir para alcançá-los com base em princípios de racionalidade e interagir com outros agentes, sejam eles artificiais ou humanos (Russell and Norvig, 2016). Esse sistema permite a divisão de tarefas complexas em subtarefas especializadas, onde cada agente pode

atuar de forma autônoma, mas coordenada, para atingir objetivos comuns.

Já os LLM são modelos avançados de aprendizado de máquina que possuem um grande número de parâmetros e passam por um extenso processo de pré-treinamento. Esse treinamento inclui tarefas como modelagem de linguagem mascarada e predição autorregressiva para aprenderem a representar o significado contextual do texto e a calcular probabilidades com base em vastos conjuntos de dados textuais.

Para que um LLM seja considerado eficaz, ele deve apresentar quatro características fundamentais (Yang et al., 2024): (i) compreensão profunda do contexto da linguagem natural; (ii) capacidade de gerar textos com qualidade semelhante à escrita humana; (iii) consciência contextual, especialmente em domínios que exigem amplo conhecimento; e (iv) habilidade avançada de seguir instruções, sendo útil para a resolução de problemas e a tomada de decisões. A partir de 2018, diversos LLM foram desenvolvidos e lançados, conquistando grande popularidade. Entre os exemplos mais notáveis estão **GPT-3** e **GPT-4** (OpenAI) (Achiam et al., 2023) (OpenAI, 2024), **PaLM** e **Gemini** (Google) (Chowdhery et al., 2023), e **LLaMA** (Meta) (Touvron et al., 2023), além de **Megatron-Turing NLG** (NVIDIA) (Smith et al., 2022), **Jurassic-1 Jumbo** (AI21 Labs) (Reed et al., 2022) e o **DeepSeek** (Bi et al., 2024), que têm se destacado como exemplos notáveis nesse campo.

Esses avanços têm ampliado as aplicações de LLM em tarefas complexas, como interação natural com usuários, análise de dados e automação de processos. Nesse contexto, surge uma oportunidade de resolver problemas complexos, combinando os LLM e SMA. A combinação dessas duas tecnologias tem recebido grande atenção recente na literatura, impulsionada pela busca por sistemas mais autônomos e capazes de tomar decisões baseadas em dados e considerando cenários dinâmicos.

Diversos trabalhos têm demonstrado como a integração de SMA e LLM pode ser aplicada com sucesso em domínios variados, desde saúde e manufatura até ambientes virtuais e gestão de fornecedores. Ho et al. (2024) propõem uma abordagem que combina SMA com LLM para automatizar a construção de grafos de conhecimento específicos para o gerenciamento de diabetes. Wang and Qin (2024), exploram a utilização de LLM, combinados com frameworks como Langchain e METAGPT, para desenvolver um sistema de regulação automática de processos industriais. O sistema proposto visa gerenciar energia e promover a redução de emissões em ambientes industriais complexos, por meio do controle preciso e da otimização das operações das linhas de produção. Outro trabalho relevante, apresentado em Wang and Ke (2024), propõe o LLM-Seg, um método em dois estágios que combina *modelos de linguagem visual* (VLM) e modelos de fundação em visão computacional para segmentação de imagens. Embora o foco desse trabalho seja na segmentação de imagens, ele adota uma estratégia semelhante à nossa proposta, dividindo o processo em etapas especializadas, como a interpretação de prompts e a execução de tarefas específicas. Zhao et al. (2024) propõem o HAS framework, um sistema multiagente hierárquico baseado em LLMs para navegação em

¹ Software desenvolvido pela empresa Deeptera

ambientes complexos no Minecraft. A arquitetura combina planejamento centralizado com execução descentralizada, comunicação dinâmica entre agentes e integração de informações multimodais. Já Li et al. (2024), propõem um sistema de suporte à decisão baseado em Gêmeos Digitais e Sistemas Multiagentes para gerenciamento sustentável e resiliente de fornecedores em manufatura inteligente. O sistema integra dinamicamente dados, modelos de gestão e tecnologias avançadas de suporte à decisão, sendo aplicado com sucesso em uma empresa alemã de bombas solares.

Embora cada estudo aborde desafios específicos de seu contexto, todos compartilham uma abordagem comum: a divisão de processos complexos em etapas especializadas, a utilização de agentes inteligentes para tarefas específicas e a integração de tecnologias avançadas para automação e otimização. Essa estratégia modular e flexível, presente em propostas como o LLM-Seg de Wang and Ke (2024) e o HAS framework de Zhao et al. (2024), ressoa diretamente com a nossa abordagem, que utiliza agentes especializados e LLM para otimizar processos industriais. Além disso, a ênfase em comunicação dinâmica, planejamento centralizado e execução descentralizada, como visto em Zhao et al. (2024) e Li et al. (2024), reforça a viabilidade da nossa proposta para resolver problemas complexos, como os presentes em ambientes industriais. Esses trabalhos oferecem insights valiosos para futuras melhorias e aplicações em cenários reais.

A integração de LLM em *Sistemas Multiagentes* (SMA) requer ferramentas e frameworks especializados que facilitem a comunicação, coordenação e tomada de decisão entre os agentes. Frameworks como Jason, JADE (Java Agent Development Framework) e PLASA são amplamente utilizados para desenvolver sistemas multiagentes tradicionais (Cardoso and Ferrando, 2021), oferecendo suporte para a criação de agentes autônomos, troca de mensagens e simulação de ambientes complexos. No entanto, esses Frameworks tipicamente não apresentam funcionalidades nativas para o uso de LLM. Recentemente, frameworks como LangChain (Topsakal and Akinci, 2023), LangGraph (LangChain, 2024), AutoGen (Wu et al., 2023) e CrewAI (CrewAI, 2024) têm ganhado destaque por permitir a integração de LLM em sistemas multiagentes. O LangChain, por exemplo, oferece ferramentas para conectar modelos de linguagem a fontes de dados externas, APIs² e outros agentes, possibilitando a criação de fluxos de trabalho complexos e interativos. LangGraph é uma estrutura construída sobre LangChain, simplificando a criação e gerenciamento de agentes e seus ambientes de execução. Ele se diferencia por oferecer loops, controle granular e memória persistente, suportando arquiteturas como single-agent, rede de agentes e hierárquico, ideal para cenários complexos. Já o AutoGen é projetado especificamente para construir sistemas multiagentes onde LLM atuam como componentes centrais, permitindo a colaboração entre agentes especializados em diferentes tarefas, como análise de dados, geração de conteúdo e tomada de decisões. O CrewAI é um framework open-source que coordena agentes de IA com operações autônomas e baseadas em regras. Ele permite definir agentes com papéis, objetivos e ferramentas específicos, suportando APIs como OpenAI e Ollama. Seu foco é em tarefas complexas, como

workflows multi-etapas e tomada de decisões que exijam comunicação entre diferentes agentes, as chamadas redes de agentes, onde todos se comunicam entre si para tomar uma decisão final.

3. SISTEMA MULTIAGENTE

O SMA proposto adota um modelo hierárquico com interações com LLM, onde um agente supervisor (*Manager*) gerencia três agentes especialistas: *Helper*, *Data Analytics* e *Optimizer* para atendimento a demandas de usuários. Neste trabalho, o SMA é integrado ao sistema Bullker para flexibilizar, facilitar e agilizar o seu uso, com segurança, nas operações de gerenciamento e otimização de carregamento e descarregamento de navios. O Bullker é um sistema Web pré-existente com interface gráfica (Frontend), conjunto de funções implementadas em API (Backend) e banco de dados relacional. Essa integração é ilustrada na Figura 1.

Cada agente possui um conjunto de instruções (prompts) e contexto associado (dados). Essas instruções e contexto são concatenadas à demanda do usuário, em formato texto, e enviadas à plataforma online de inteligência artificial generativa OpenAI (modelo GPT-4o), que processa e retorna para o SMA a resposta.

O agente *Manager* é quem interage diretamente com os usuários e suas demandas. O processo de atendimento às demandas segue a seguinte ordem: (1) a demanda chega para o *Manager* em formato de texto escrito pelo usuário, a partir de requisição via interface conversacional; (2) o *Manager* determina qual agente especialista deve processá-la; (3) o agente alocado executa a tarefa atribuída e, (4) retorna os resultados ao *Manager*, que, por sua vez, (5) os entrega ao usuário.

O agente *Data Analytics* é responsável pela análise de dados e construção automatizada de gráficos, proporcionando uma visualização personalizada com base na demanda do usuário. O agente *Optimizer* invoca métodos de otimização (solver e modelo matemático), disponíveis no sistema Bullker, para sequenciamento de carregamento e descarregamento de navios, construindo planos operacionais exequíveis, quando viável. O agente *Helper* é especializado em atender demandas gerais e operacionais relacionadas ao uso do sistema convencional. O agente *Helper* executa ações no sistema Bullker de forma automatizada e ágil, como cadastros, navegação e suporte ao usuário sobre as dúvidas relacionadas ao uso do sistema.

O SMA foi construído utilizando os frameworks LangChain e LangGraph. O framework LangChain tem se mostrado robusto, com muitas funcionalidades, implementação simplificada e documentação acessível. O framework LangGraph se mostrou mais adequado para o cenário em questão em comparação com outras ferramentas, como AutoGen e CrewAI, pela capacidade em proporcionar um modelo hierárquico consistente entre os agentes, permitindo que o supervisor coordene a interação. Além disso, o LangGraph assegura que os agentes especialistas não interajam diretamente entre si, mas sempre respondam ao supervisor, que é o ponto de contato com o usuário.

² Do inglês: Application Programming Interface.

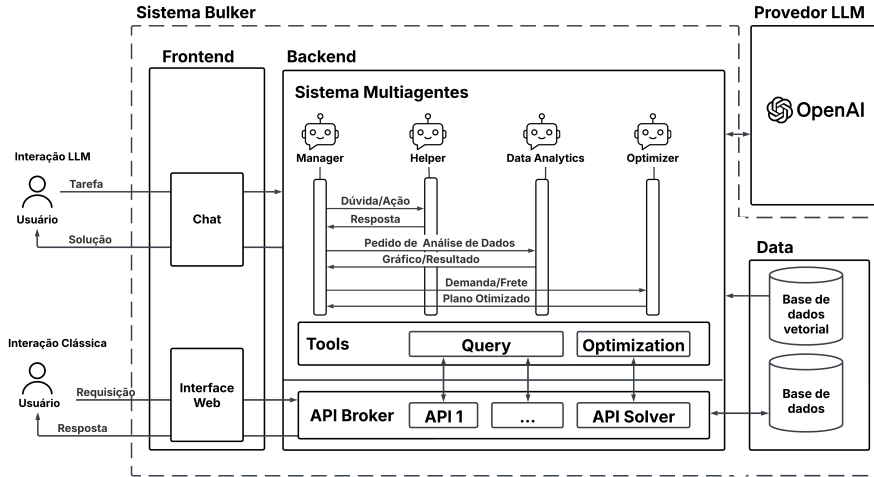


Figura 1. Sistema multiagente proposto e integrado.

3.1 Sensores e Atuadores

Além da interação controlada com o usuário, os agentes interagem com o sistema Bullker por meio de funções específicas, geralmente chamadas de *tools*. Essas funções atuam como sensores e atuadores, permitindo que os agentes percebam e interajam com o ambiente. Elas se comunicam com as APIs disponíveis (Broker) no sistema Bullker, possibilitando o acesso e a manipulação de dados no banco de dados. Isso inclui o cadastro de novos registros, a consulta de informações para geração de relatórios e gráficos, além da execução do método de otimização de sequenciamento. Essencialmente, duas funções estão disponíveis:

- *queryTools*: permite que os agentes executem código dentro do sistema Bullker. O agente *Helper* utiliza essa ferramenta para operações de inserção, atualização, exclusão e seleção de dados, enquanto o *Data Analytics* a emprega para busca de dados e geração de gráficos.
- *optimizationTools*: permite executar a otimização dos planos de carregamento e descarregamento.

3.2 Base de Conhecimento

Os agentes são contextualizados com informações de diferentes bases de conhecimento, para que possam tomar decisões que estejam de acordo com as demandas que lhes são apresentadas. A seguir é detalhada cada base de conhecimento.

Manual do Usuário Contém as diretrizes sobre como utilizar a aplicação convencional do ponto de vista do usuário, incluindo a navegação entre telas, o cadastro de informações e a interpretação dos resultados apresentados. Essa base de conhecimento é utilizada pelo agente *Helper*, que auxilia os usuários dentro do sistema.

Documentação Contém as regras de negócio do sistema, com detalhes dos fluxos operacionais e as diretrizes para o funcionamento correto da aplicação. Os agentes *Helper* e *Optimizer* utilizam essa base para orientar o usuário na configuração correta das operações e na execução da otimização dos planos de sequenciamento.

Modelo de Dados Define a estrutura do banco de dados da aplicação convencional, detalhando entidades, relações e restrições. Essa base de conhecimento permite que o *Data Analytics* faça consultas estruturadas para recuperação de informações e gere relatórios, e também que o *Helper* faça operações de inserção e consulta no banco de dados. Foram disponibilizados, junto ao modelo de dados, exemplos de consultas para serem usadas como base.

Metodologia Extreme Presentation O SMA considera a metodologia *Extreme Presentation*, que define o tipo de gráfico mais apropriado para situações específicas. Por exemplo, o gráfico de linhas tende a ser mais adequado que o gráfico de barras quando se tem mais amostras de dados (Presentation, 2025). O agente *Data Analytics* utiliza essa metodologia para definir e construir gráficos informativos e dashboards analíticos, facilitando a interpretação dos dados pelo usuário final.

Os conhecimentos indicados nesta seção foram estruturados em uma representação vetorial de textos, por meio de recursos disponíveis no framework LangChain. Adotou-se a biblioteca *Faiss* (Douze et al., 2025) para as buscas por similaridade dos dados relevantes, visando atender à demanda do usuário.

3.3 Agentes

Nesta seção são apresentados os comportamentos dos agentes *Manager*, *Helper*, *Data Analytics* e *Optimizer*. Essencialmente, os agentes seguem instruções que foram construídas por meio de técnica de engenharia de prompt iterativa, refinando continuamente os prompts com base na comparação entre respostas geradas e esperadas. O processo considerou a definição dos papéis dos agentes, restrições explícitas para evitar respostas indesejadas e o uso de contexto relevante para aumentar a assertividade. Utilizou-se também a técnica de Prompt Chaining, que consiste em quebrar tarefas em subtarefas (prompting-guide.ai, 2025).

As instruções são enviadas em tempo real ao modelo GPT-4o, da OpenAI, com um parâmetro de temperatura (0.1) que define a aleatoriedade das respostas (Renze and Gu-

ven, 2024). Valores próximos a um (1) indicam capacidade de criatividade nas respostas, enquanto que valores próximos a zero (0) indicam estabilidade nas respostas. Adotou-se o valor um (1) para temperatura, pois esse valor levou os agentes a terem melhor interação com as funções (tools) e base de dados, gerando respostas precisas. Ao experimentar valores menores, o agente teve dificuldade em absorver o contexto das interações anteriores com os usuários, levando-o a alucinações e imprecisões. O código-fonte de implementação dos agentes e seus prompts completos estão disponíveis em <https://github.com/Deeptera/SMA>.

Os seguintes elementos de cada agente são detalhados: (1) as entradas enviadas aos agentes, (2) um resumo das instruções (prompts) que os agentes seguem, (3) os processamentos de funções (tools) realizados pelo agente no ambiente (sistema Bullker), e os de saídas entregue/realizadas pelos agentes para atendimento à demanda de usuário.

Agente Manager Agente de nível mais alto na hierarquia que atua como coordenador central de tarefas.

Algoritmo 1 Agente *Manager*

Entradas:

- Mensagem do Usuário e histórico de interações.

Resumo das instruções:

- [1] Você é o *Manager*, o assistente virtual da aplicação.
- [2] Você gerencia três especialistas: *Helper*, *Data Analytics* e *Optimizer*. Ao receber uma tarefa, a encaminhe ao especialista apropriado.
- [3] Para tarefas que envolvam cadastro/alteração de dados do sistema ou dúvidas sobre a aplicação, encaminhe para o *Helper*.
- [4] Para gerar relatórios e gráficos sobre planos de sequenciamento ou outros dados do sistema, encaminhe para o *Data Analytics*.
- [5] Para guiar o usuário até a otimização de sua demanda ou otimizar sequências de carregamento ou descarregamento, consulte o *Optimizer*.
- [6] Após receber a resposta do agente especialista reencaminhe-a ao usuário.

Saída:

- Resposta do especialista (texto e/ou imagem).

Agente Helper Responsável por ajudar os usuários a navegar na plataforma, responder perguntas sobre o funcionamento do site e cadastrar ou alterar dados no banco de dados conforme solicitado.

Algoritmo 2 Agente *Helper*

Entrada:

- Conhecimento do Modelo de Dados.
- Conhecimento da Documentação.
- Conhecimento do Manual do Usuário.
- Dúvida do usuário ou ação desejada.

Resumo das instruções:

- [1] Você é um ajudante virtual.
- [2] Seu papel é ajudar os usuários a navegar na plataforma e cadastrar/alterar dados no banco.
- [3] Para responder perguntas que envolvam navegabilidade, cadastro de informações ou os métodos de otimização, consulte a base de conhecimento.
- [4] No caso de inserir ou alterar dados do banco, gere a query em Django ORM, e passe para a ferramenta queryTools somente a query criada, para ela ser executada.

Processamento:

- Agente executa o código-fonte python criado usando queryTools.

Saída:

- Resposta para a dúvida solicitada pelo usuário ou resultado da ação realizada.

Agente Data Analytics É responsável por ajudar o usuário a visualizar relatórios e gráficos com base nos dados do banco.

Algoritmo 3 Agente *Data Analytics*

Entrada:

- Conhecimento do Modelo de Dados.
- Conhecimento da Metodologia *Extreme Presentation*.
- Pedido de Análise de Dados.

Resumo das instruções:

- Você deve gerar um código em Python que contenha:
- [1] Uma query Django ORM para obter os dados necessários, convertendo-os para os tipos numéricos adequados.
- [2] O trecho de código que utiliza Seaborn para gerar um gráfico completo e bem formatado.
- [3] A conversão do gráfico em bytes e o salvamento no banco de imagens.

Processamento:

- Agente executa o código-fonte python criado usando queryTools.

Saída:

- Análise visual (gráfico) para a demanda do usuário.

Agente Optimizer É responsável por atender a demandas relativas à funcionalidade de otimização do sistema convencional.

Algoritmo 4 Agente *Optimizer*

Entrada:

- Conhecimento do Modelo de Dados.
- Conhecimento da Documentação.
- Demanda/Frete a ser otimizado.

Resumo das instruções:

- [1] Você é um especialista em otimização de sequenciamento para carregamento e descarregamento de navios graneleiros.
- [2] Na posse do nome do plano, execute a otimização com a ferramenta *optimizationTool*.
- [3] Ao final da otimização, incluir a duração total do planos e o número de sequências geradas na resposta.

Processamento:

- Agente executa o método de otimização, por meio do *optimizationTools*, e salva resultados.

Saída:

- Cadastro dados necessários para executar o método de otimização.
- Plano otimizado de sequenciamento de carregamento/d Descarregamento.
- Resumo do plano otimizado (duração e número de sequências).

4. CASO DE USO

Nesta seção é apresentada uma jornada de interações do usuário com o sistema SMA proposto, destacando as funcionalidades dos agentes especializados e sua integração para atender às demandas do usuário. A interação sempre começa com o **agente Manager**, apresentado como “Assistente” ao usuário.

Durante as interações realizadas, observou-se que o sistema pode apresentar instabilidade nas respostas: uma mesma pergunta pode ser respondida de formas distintas em diferentes momentos, dependendo da interpretação e da temperatura do agente. Também houveram casos de alucinações, nos quais as respostas geradas não atenderam ao objetivo da consulta ou continham informações inventadas, exigindo validação pelo usuário ou iterações adicionais. As subseções a seguir exemplificam alguns cenários, nos quais nenhum comportamento inesperado foi identificado.

4.1 Solicitação de Informação e Cadastro

Ilustrado na Figura 2, o usuário solicita informações sobre como cadastrar um novo navio no sistema. O **agente Helper** responde de forma clara e direta, explicando o processo e se oferecendo para realizar o cadastro. Essa interação elimina a necessidade de o usuário buscar as informações ou entender detalhes técnicos do sistema. Após o envio dos dados pelo usuário, o agente gera e executa o algoritmo para cadastrar o navio no banco de dados com sucesso. O agente também é capaz de atender a solicitações semelhantes, como a atualização de dados ou a exclusão de registros, para qualquer entidade do sistema.

4.2 Gerar um Plano Otimizado

Uma vez que o operador logístico já cadastrou os dados do navio, o processo de definição da sequência ótima de carregamento ou descarregamento do navio pode ser iniciado. É

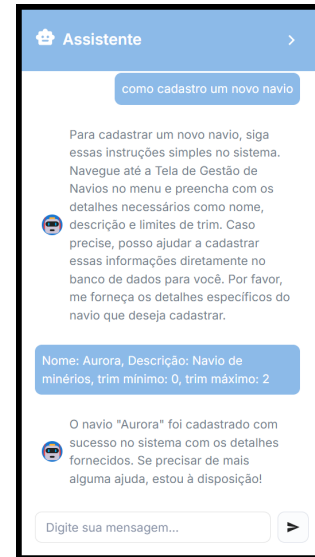


Figura 2. Busca de informação de cadastro com o agente *Helper*.

comum que essa tarefa seja feita manualmente pelo capitão do navio ou operador do porto e sua equipe, quando não há um sistema como o Bullker à disposição. Para agilizar o processo, na Figura 3 o usuário solicita a execução de um plano otimizado. O **agente Optimizer** pergunta o nome do plano, que nesse caso já havia sido configurado, e o agente executa as funções de otimização do sistema, gerando um plano otimizado com duração prevista de 111.63 horas em um total de 19 sequências (número de trocas entre máquinas e porões). As informações ficam disponíveis no sistema Bullker para consulta. Esse processo reduz o tempo de operação e auxilia o capitão do navio, responsável legal pela embarcação.

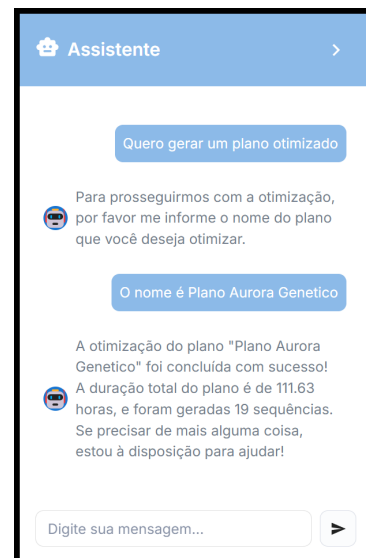


Figura 3. Otimização executada pelo agente *Optimizer*.

4.3 Analisar Planos

Análises de dados das operações de embarque e desembarque são desejáveis em várias etapas na indústria naval, como nas fases de planejamento e operação. Nesse sentido,

o usuário pode solicitar a geração de gráficos diversos ao sistema. Com o plano otimizado gerado anteriormente, o usuário pode compará-lo com planos já existentes no frete utilizado, sejam eles feitos manualmente pelo capitão ou otimizados pelos diferentes algoritmos do sistema Bullker. No cenário em análise, o **agente *Data Analytics*** é acionado para criar um gráfico personalizado que mostra a duração de cada um dos planos já existentes. O agente processa os dados, gera o gráfico e o disponibiliza de forma clara e intuitiva para o usuário, como ilustrado na Figura 4 e Figura 5. Entretanto, em alguns casos, o agente exibia dados inexistentes, especialmente para demandas complexas ou mal formuladas.

Com o agente, o algoritmo necessário para extrair a informação do banco de dados e gerar o gráfico é construído dinamicamente, para qualquer demanda do usuário. Em um sistema convencional, o desenvolvedor seria responsável por criar uma função de geração de gráfico para cada demanda possível, tornando o processo inflexível.

Na Figura 5 é possível ver a comparação de três planos de descarregamento. O plano manual gasta 132 horas, enquanto os planos otimizados, utilizando o Solver SCIP e um algoritmo Genético, são executados com 15,42% de tempo a menos.

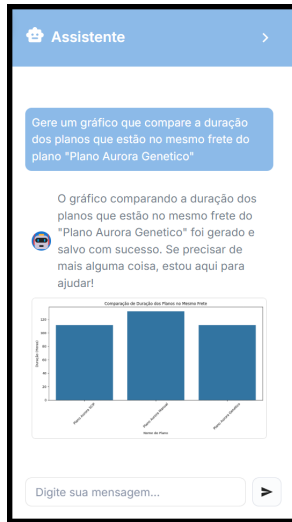


Figura 4. Dashboard analítico gerado pelo agente *Data Analytics*.

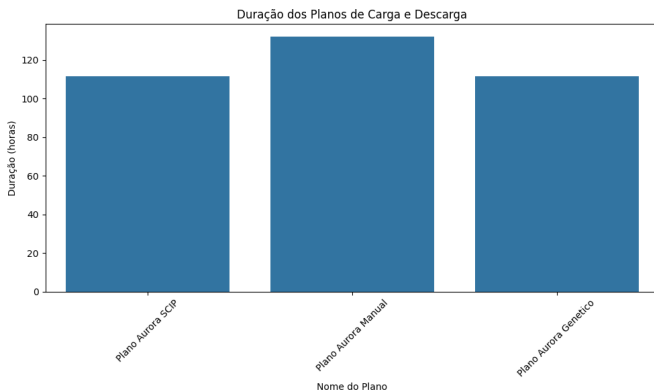


Figura 5. Gráfico gerado pelo agente *Data Analytics*.

5. CONCLUSÃO

Este trabalho apresentou a proposta e implementação de um Sistema Multiagente baseado em LLM para melhorar a interação do usuário com um sistema convencional de gerenciamento e otimização de carregamento e descarregamento de navios, com foco em automação, eficiência e flexibilidade. A arquitetura proposta, composta por quatro agentes especializados (*Manager*, *Helper*, *Optimizer* e *Data Analytics*), mostra-se uma solução viável para os desafios da logística naval, integrando-se de forma eficaz ao sistema convencional.

O caso de uso abordado analisou e demonstrou a capacidade do sistema em atender a demandas complexas, desde o cadastro de navios até a geração de planos otimizados e a criação de análises visuais personalizadas. A interação fluida entre os agentes, mediada pelo agente *Manager*, permitiu uma experiência inovadora para o usuário, diferente da proporcionada por aplicações tradicionais, enquanto a customização proporcionada pela biblioteca LangGraph proporcionou mais liberdade para os agentes, que não ficam presos a um fluxo estruturado de tarefas sequenciais. Essa abordagem traz ganhos como a **agilidade** na execução de tarefas complexas, a **automação** de processos que antes demandavam intervenção manual, a **facilidade de uso** para usuários sem conhecimento técnico e a geração de **insights rápidos** por meio de análises visuais e comparativas.

É importante considerar os trade-offs envolvidos na proposta. O uso de modelos como o GPT-4o, embora traga benefícios significativos em termos de precisão e capacidade de processamento de linguagem natural, implica em custos financeiros associados à utilização da API da OpenAI. Esses custos podem variar dependendo do volume de requisições e da complexidade das tarefas executadas. Além disso, a dependência de uma API externa pode introduzir questões relacionadas à latência e à disponibilidade do serviço.

Como trabalhos futuros, sugere-se a expansão do sistema para incluir novas funcionalidades e agentes especializados, com foco na minimização de problemas de instabilidade e alucinações, e reforço da segurança da informação, com a criação de um agente de Segurança dedicado. Este agente atuaria como *guard rail*, analisando semanticamente prompts e respostas para mitigar ataques como *prompt injection* (Liu et al., 2023), controlando permissões de acesso e garantindo conformidade com normas. Integrado aos sistemas de autenticação, filtraria solicitações críticas, permitindo apenas operações autorizadas conforme o perfil do usuário.

Outra direção promissora é a adoção de Small LLMs (modelos de linguagem menores e locais) como alternativa a LLMs pagos como GPT-4, reduzindo custos e aumentando a autonomia do sistema em ambientes com conectividade limitada. Embora exijam infraestrutura dedicada para hospedagem, valeria investigar o desempenho comparativo de modelos como Claude, Gemini e soluções open-source em tarefas específicas de gerenciamento naval.

REFERÊNCIAS

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Bi, X., Chen, D., Chen, G., Chen, S., Dai, D., Deng, C., Ding, H., Dong, K., Du, Q., Fu, Z., et al. (2024). Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*.
- Cardoso, R.C. and Ferrando, A. (2021). A review of agent-based programming for multi-agent systems. *Computers*, 10(2), 16.
- Chen, W., Yan-yi, L., Tie-zheng, G., Da-peng, L., Tao, H., Zhi, L., Qing-wen, Y., Hui-han, W., and Ying-you, W. (2024). Lsystems engineering issues for industry applications of large language model. *Applied Soft Computing*, 151, 111165.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al. (2023). Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240), 1–113.
- CrewAI (2024). Crewai: Framework for multi-agent collaboration. URL <https://www.crewai.com/>. Acessado em: 2-28-2025.
- Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.E., Lomeli, M., Hosseini, L., and Jégou, H. (2025). The faiss library. URL <https://arxiv.org/abs/2401.08281>.
- Hassan, M.A., Zardari, S., Farooq, M.U., Alansari, M.M., and Nagro, S.A. (2024). Systematic analysis of risks in industry 5.0 architecture. *Applied Sciences*, 14(4), 1466.
- Ho, D.H., Das, U., Ho, R., and Lee, Y. (2024). Leveraging multi-agent systems and large language models for diabetes knowledge graphs. In *2024 IEEE International Conference on Big Data (BigData)*, 3401–3410. IEEE.
- LangChain (2024). Langgraph: Multi-agent workflows with llms. URL <https://langchain-ai.github.io/langgraph/>. Acessado em: 2-28-2025.
- Lasi, H., Fettke, P., Kemper, H.G., Feld, T., and Hoffmann, M. (2014). Industry 4.0. *Business & information systems engineering*, 6, 239–242.
- Leng, J., Sha, W., Wang, B., Zheng, P., Zhuang, C., Liu, Q., Wuest, T., Mourtzis, D., and Wang, L. (2022). Industry 5.0: Prospect and retrospect. *Journal of Manufacturing Systems*, 65, 279–295.
- Li, Y., Tsang, Y.P., Wu, C.H., and Lee, C.K.M. (2024). A multi-agent digital twin-enabled decision support system for sustainable and resilient supplier management. *Computers & Industrial Engineering*, 187, 109838.
- Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., et al. (2023). Prompt injection attack against llm-integrated applications. *arXiv preprint arXiv:2306.05499*.
- OpenAI (2024). Hello gpt-4o. URL <https://openai.com/index/hello-gpt-4o/>. Accessed: 2025-03-14.
- Paschek, D., Luminosu, C.T., and Ocakci, E. (2022). Industry 5.0 challenges and perspectives for manufacturing systems in the society 5.0. *Sustainability and Innovation in Manufacturing Enterprises: Indicators, Models and Assessment for Industry 5.0*, 17–63.
- Presentation, E. (2025). Extreme presentation. <https://extremepresentation.com/>. Acessado em: 2-28-2025.
- promptingguide.ai (2025). Prompt chaining. https://www.promptingguide.ai/techniques/prompt_chaining. Acessado em: 3-14-2025.
- Reed, L., Li, C., Ramirez, A., Wu, L., and Walker, M. (2022). Jurassic is (almost) all you need: Few-shot meaning-to-text generation for open-domain dialogue. In *Conversational AI for Natural Human-Centric Interaction: 12th International Workshop on Spoken Dialogue System Technology, IWSDS 2021, Singapore*, 99–119. Springer.
- Renze, M. and Guven, E. (2024). The effect of sampling temperature on problem solving in large language models. URL <https://arxiv.org/abs/2402.05201>.
- Russell, S.J. and Norvig, P. (2016). *Artificial intelligence: a modern approach*. pearson.
- Smith, S., Patwary, M., Norick, B., LeGresley, P., Rajbhandari, S., Casper, J., Liu, Z., Prabhunoye, S., Zerveas, G., Korthikanti, V., et al. (2022). Using deepspeed and megatron to train megatron-turing nl 530b, a large-scale generative language model. *arXiv preprint arXiv:2201.11990*.
- Topsakal, O. and Akinci, T.C. (2023). Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences*, volume 1, 1050–1056.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Wang, B., Zhou, H., Li, X., Yang, G., Zheng, P., Song, C., Yuan, Y., Wuest, T., Yang, H., and Wang, L. (2024). Human digital twin in the context of industry 5.0. *Robotics and Computer-Integrated Manufacturing*, 85, 102626.
- Wang, J. and Ke, L. (2024). Llm-seg: Bridging image segmentation and large language model reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 1765–1774.
- Wang, Z. and Qin, H. (2024). Intelligent industrial production process automatic regulation system based on llm agents. In *2024 5th International Conference on Artificial Intelligence and Electromechanical Automation (AIEA)*, 133–137. IEEE.
- Wooldridge, M. (2009). *An introduction to multiagent systems*. John wiley & sons.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al. (2023). Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.
- Yang, J., Jin, H., Tang, R., Han, X., Feng, Q., Jiang, H., Zhong, S., Yin, B., and Hu, X. (2024). Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, 18(6), 1–32.
- Zhao, Z., Chen, K., Guo, D., Chai, W., Ye, T., Zhang, Y., and Wang, G. (2024). Hierarchical auto-organizing system for open-ended multi-agent navigation. *arXiv preprint arXiv:2403.08282*.